

Łańcuchy znaków

1. Typ znakowy char

Typ znakowy **char** przechowuje wartości liczbowe całkowite z przedziału od -128 do +127. Domyślnie typ służy do przechowania kodów znaków z tabeli ASCII.

Wydruk zmiennej tego typu, wymusza wydruk znaku o takim właśnie kodzie (z tablicy ASCII).

Aby nie musieć pamiętać numeru każdego znaku, można taki znak wpisać bezpośrednio z użyciem apostrofu. **Pojedyncze znaki obsługujemy z użyciem apostrofa.**

```
char z1 = 'q';
```

```
char z2 = 'a';
```

```
char z3 = 'p';
```

```
char z4 = 't';
```

```
char z5 = 's';
```

```
cout << z1 << z2 << z3 << z4 << z5 << "\n";
```

2. Tablica znaków char[]

Znaki można także umieszczać w tabeli znaków (a nie każdy znak w osobnej zmiennej). Ma to swoje zalety, bo w jednej tablicy można zmieścić cały tekst (tylko jeden element do wydruku).

Tablica musi być co najmniej o jeden większa niż liczba znaków w niej zawarta!

Próba zapisania tekstu do tablicy o niewystarczającym rozmiarze najprawdopodobniej zakończy się błędem kompilacji.

Tablica może być (znacznie) większa niż ilość znaków jaką potrzebujemy. Nie spowoduje to żadnego błędu (jedynie zmarnujemy trochę pamięci). Jeśli znamy rozmiar tekstu docelowego, to wielkość tablicy znaków możemy dobrać idealnie. Jeśli nie znamy potencjalnego tekstu, to rozmiar tablicy powinien być dobrany „optymalnie” (aby nie był za mały, a jednocześnie nie marnowało się nie wiadomo ile pamięci).

Należy pamiętać, że w takiej tablicy, za ostatnim znakiem, pojawia się dodatkowy znak końca '\0', aby było wiadomo, gdzie tekst się kończy (np. w którym miejscu należy przestać go drukować).

Tablicę znaków drukujemy jak zwykłą zmienną, podając jej nazwę. Kolejne znaki są drukowane po kolei, aż do napotkania znaku końca '\0'.

// Tak naprawdę, to ostatniego znaku '\0' nie musimy pisać. // Zostanie tam wstawiony automatycznie.

```
char tekst[6] = {'M', 'a', 'p', 'i', 's', '\0'}; cout << tekst << "\n";
```

3. Ustawienie tekstu w tablicy znaków char[]

Deklarujemy tablicę znaków o znanym rozmiarze.

Elementy ustawiamy ręcznie, każdy z osobna (niczym w tablicy). Tak naprawdę ostatniego elementu ('\0') nie trzeba by ustawiać, bo podobnie jak w tablicy liczb, pozostałe (nieustawione) komórki są zerowane. `char tekst1[6] = {'M', 'a', 'p', 'i', 's', '\0'}`
};

Deklarujemy tablicę znaków o znanym rozmiarze.

Podstawiamy do komórek tablicy tekst. Każdy kolejny znak będzie zapisany w kolejnej komórce tablicy. Teksty zapisujemy z użyciem cudzysłowu.

```
char tekst2[6] = "Napisi";
```

Deklarujemy tablicę znaków. Potrzebny rozmiar obliczy się automatycznie na podstawie długości tekstu jaki próbujemy ustawić (zostanie uwzględniony także znak końca '\0').

```
char tekst3[] = "Napisi";
```

Dobra zasada programistyczna. Tablice znaków, które mają być niezmiennie przez cały czas trwania programu (funkcji) powinny być blokowane modyfikatorem **const**, aby uniknąć ich (przypadkowej) modyfikacji. `const char tekst4[] = "Napisi";`

Umieszczanie znaku końca '\0' wewnątrz tablicy znaków powoduje, że taki tekst będzie drukowany jedynie do tego znaku. Tym sposobem możemy przechowywać w tablicy dodatkowe informacje (nie wiem po co w taki sposób, ale można).

```
char tekst5[] = "Zabie programowac.\0... sklamalem. :)"; cout << tekst5 << "\n";
```

4. Wczytanie do tablicy znaków char[]

Do tablicy znaków **char[]** możemy pobrać wartości z klawiatury.

Zapis poniżej pobiera nam tekst z wejścia, aż do napotkania znaku białego (enter, tabulator, spacja, itp.). Tym samym nie wczytamy zdania zawierającego kilka wyrazów, a jedynie pierwszy wyraz (czyli ciąg znaków

nieoddzielony znakiem białym).

```
char bufor[ 100 ];  
  
cin >> bufor;  
cout << bufor << "\n";
```

Można także wymusić, aby tekst był wczytywany z wejścia aż do napotkania konkretnego znaku (w przykładzie **entera**). Tym samym można pobrać zdanie zawierające kilka wyrazów.

Do tego celu służy funkcja **getline()**. Uwaga! Funkcja musi być wywołana z wejścia (**cin** z kropką z przodu, a jej parametry, to bufor do jakiego zapisujemy tekst oraz maksymalna liczba znaków jaką chcemy zapisać w buforze, wartość ta jednak nie może być większa niż rozmiar tablicy `char[]`).

```
char bufor[ 100 ];  
  
cin.getline( bufor, 100 );  
  
cout << bufor << "\n";
```

Uwaga! Jeśli wcześniej, przed **getline()** wczytywaliśmy cokolwiek z użyciem `cin`, to na wejściu pozostał biały znak lub enter, który spowoduje, że nie pobierzemy całej linii tekstu (bo wczytujemy do enteru, a skoro enter został w strumieniu, to tylko do niego wczytamy = nic nie wczytamy). Aby tego uniknąć, to **przed użyciem getline()** zapisujemy:

aby zignorować ostatni znak pozostawiony w strumieniu (domyślnie, ten enter co miał pozostać).

```
cin.ignore();
```

lub całkowicie czyścimy strumień, ze wszystkich pozostawionych znaków. Obie instrukcje muszą być użyte.

```
cin.clear();
```

```
cin.sync();
```

5. Operacje na tekście

Tablica znaków `char[]` jest to zwyczajna tablica. Można na jej elementach (znakach) wykonywać operacje niczym na elementach tablicy liczbowej.

Uwaga! Tablica operuje na indeksach – numerach komórek, pierwszy znak w tablicy ma indeks **zero**.

Jeśli nie znamy rozmiaru tablicy (tekstu), to musimy pamiętać, że tekst

kończy się na znaku końca '\0'.

Operacje **na pojedynczych znakach** wykonujemy z wykorzystaniem **apostrofów**.

```
char bufer[] = "Bardzo długie zdanie do testów wydruku";
```

```
bufer[5] = 'X';
```

```
bufer[11] = 'P';
```

```
bufer[16] = '\0';
```

```
cout << bufer << "\n";
```

6. Tablica znaków ASCII

Podstawowe znaki i symbole drukowane, znaki specjalne, kody sterujące, są ustandaryzowane w tablicy znaków ASCII.

0 - znak końca '\0'

1 – 31 – znaki specjalne, kody sterujące (np. enter)

32 – 125 – znaki i symbole drukowane

126 – tylda. Nie tyle symbol, co instrukcja specjalna (z użyciem tyldy możemy drukować np. polskie znaki. Taki przodek prawego klawisza Alt).

127 – Delete

128-255 (lub -128-0, zależy jak interpretujemy **signed/unsigned char**). Znaki/symbole różne w zależności od aktywnej strony kodowej. Mogą tam być np. polskie znaki, cyrylica, niemieckie umlauty, „krawężniki” czy inne „krzaczk”.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

7. Konwertery tekstowo-liczbowe (dla typu char[])

W bibliotekach są dostępne funkcje, które konwertują tekst na liczby i odwrotnie.

Wymagane dołączenie biblioteki:

#include <stdlib.h>

```
char text_int[] = "12345";
```

```
char text_float[] = "12.34";
```

```
// ASCII to integer
```

```
int val_int = atoi(text_int);
```

```
// ASCII to float (właściwie to double)
```

```
float val_float = atof(text_float);
```

```
cout << val_int << "\n";
```

```
cout << val_float << "\n";
```

```
int val_int = 100;
```

```
float val_float = 78.99;
```

```

// Tablica znaków musi być na tyle duża,
// aby wszystkie cyfry się z niej zmieściły. char text_int[12];
char text_float[12];

// Int to ASCII
// (wartość liczbową, bufor tekstowy,
// system liczbowy na jaki konwertujemy)
itoa(val_int, text_int, 10);

// (wartość liczbową, dokładność, bufor tekstowy)
gcvt(val_float, 6, text_float);

cout << text_int << "\n";
cout << text_float << "\n";

```

8. Typ string

Mamy także dostęp do typu **string**. Jest to typ **dynamiczny** - doskonały, inteligentny, sam się rozszerza i kurczy w miarę potrzeby. Nie trzeba deklarować rozmiaru w żaden sposób.

Deklaracja typu i wydruk:

```

string napis = "Alo ma koba";

cout << napis << "\n";

```

Wczytanie z klawiatury (wczytamy tylko jeden wyraz, pobranie znaku zakończy się na znaku białym – spacji, tabulatorze, enterze, itp.)

```

string napis;

cin >> napis;

cout << napis << "\n";

```

Wczytanie z klawiatury ciągu znaków (musimy użyć funkcji **getline()**):

```

string napis;

getline(cin, napis);

cout << napis << "\n";

```

Uwaga! Jeśli wcześniej, przed **getline()** wczytywaliśmy cokolwiek z użyciem **cin**, to na wejściu pozostał biały znak lub enter, który spowoduje, że nie pobierzemy całej linii tekstu (bo wczytujemy do enteru, a skoro enter został w strumieniu, to tylko do niego wczytamy = nic nie wczytamy). Aby tego uniknąć, to **przed użyciem getline()** zapisujemy:

aby zignorować ostatni znak pozostawiony w strumieniu (domyślnie, ten enter co miał pozostać).

```
cin.ignore();
```

lub całkowicie czyścimy strumień, ze wszystkich pozostawionych znaków. Obie instrukcje muszą być użyte.

```
cin.clear();
```

```
cin.sync();
```

Typ string to także jest tablica. Można na nim wykonywać te same operacje (jeśli chodzi o dostęp do poszczególnych znaków) co na typie char[].

```
string txt = "Ala ma kota";
```

```
txt[4] = 'i';
```

```
txt[8] = '\t';
```

```
cout << txt << "\n";
```

9. Konwertery tekstowo-liczbowe (dla typu string)

Uwaga! Wymagane włączenie standardu kompilacji C++11 aby poniższe zadziałało.

Wymaga dołączenia biblioteki:

```
#include <string>
```

```
string str_int = "12345";
```

```
string str_float = "12.34";
```

```
// string to integer
```

```
int val_int = stoi(str_int);
```

```
// string to float
```

```
float val_float = stof(str_float);
```

```
// stod() - dla double
```

```
cout << val_int << "\n";
```

```
cout << val_float << "\n";
```

```
int val_int = 100;
```

```
float val_float = 78.99;
```

```
string text_int;
```

```
string text_float;
```

```
text_int = to_string(val_int);
```

```
text_float = to_string(val_float);
```

```
cout << text_int << "\n";
cout << text_float << "\n";
```

10. Różne przydatne funkcje (dla typu string)

Typ string posiada mnóstwo ciekawych funkcji. Funkcje są wywoływane z obiektu (stąd ta kropka). Poniżej zaprezentowano tylko najważniejsze z nich.

length() - zwraca ilość znaków.

```
string txt = "Ala ma kota";
cout << txt.length() << endl; // 11 znaków
```

find() - znajduje pojedyncze znaki, bądź całe ciągi znaków. Zwraca **indeks** komórki, gdzie wystąpił znak (lub początek podciagu). Zwraca **-1** (lub 4'294'967'295 jeśli unsigned), gdy nie znajdzie niczego.

```
string txt = "Ala ma kota";
cout << txt.find('k') << endl; // 7
cout << txt.find("ma") << endl; // 4
```

substr() - zwraca podciąg tekstowy, zakres sterowany parametrami.

```
string txt = "Ala ma kota";
cout << txt.substr(4, 5) << endl; // ma ko
```

swap() - zamienia miejscami dwa teksty, dwóch różnych zmiennych.

```
string napis = "Pawel ma psa";
string txt = "Ala ma kota";
napis.swap(txt);
cout << napis << endl;
cout << txt << endl;
```

append() - dołącza tekst do istniejącego tekstu (na jego koniec):

```
string txt = "Ala ma kota";
txt.append(" i psa");
cout << txt << endl;
```

compare() - porównuje dwa teksty, zwraca 0, gdy oba są identyczne, zwraca -1, gdy lewy jest „mniejszy” (tj. byłby wcześniej gdybyśmy posortowali te teksty). Zwraca 1, gdy prawy jest mniejszy.

```
string txt = "Ala";
string napis = "Kasia";
cout << txt.compare(napis) << "\n";
```

Uwaga! Standard C++11 umożliwia niektóre operacje bez korzystania z powyższych funkcji, np.:

Porównywanie stringów – bez potrzeby użycia compare():

```
string txt = "Ala";
string napis = "Kasia";
if( txt != napis )
```


Konkatenacja (łączenie tekstów) – bez potrzeby użycia `append()`:

```
string text = "Ala";
```

```
text += " kota";
```

```
text += " i psa";
```

```
text += " i chomika";
```

```
cout << text << endl;
```